

BERT 는 어떻게 생겼는가

모델 구조와 입력 표현

"BERT 는 사실, Transformer Encoder 를 깊게 쌓은 것이다."

● BERT-Base vs BERT-Large

두 가지 사이즈로 발표된 모델 스펙

BERT-Base

GPT 와 동일 크기 (통제 변인)

L (Layers)	12
H (Hidden)	768
A (Heads)	12
FFN (4H)	3,072

110M

총 파라미터

BERT-Large

더 깊고 더 넓은 모델

L (Layers)	24
H (Hidden)	1,024
A (Heads)	16
FFN (4H)	4,096

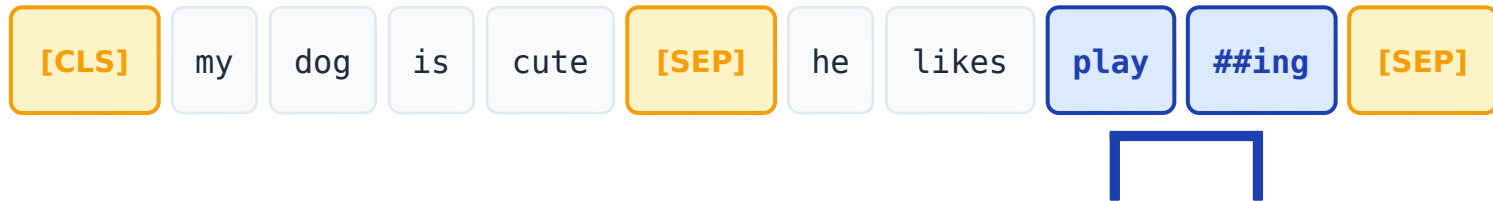
340M

총 파라미터

핵심 : BERT 는 새 연산이 아니라 , 검증된 Transformer Encoder 를 깊고 넓게 쌓은 것

● 입력 표현 — 시작은 이상한 분해부터

Figure 2 의 예시 문장을 보면 뭔가 어색합니다



playing → play + ##ing

? 왜 "playing" 이 "play" 와 "##ing" 으로 쪼개졌을까 ?

"##" 표시 = " 앞 토큰과 붙는 subword" 라는 의미 — 다음 슬라이드에서 자세히

● WordPiece 토큰나이저

단어가 아닌 'subword' 단위로 자르는 이유 3 가지

BERT vocabulary 크기 = **30,000 개** · 서브워드 표시 = **##** (앞 토큰과 붙는 조각)

1

OOV 문제 해결

Out-of-Vocabulary

영어 단어는 사실상 무한 . 자주 등장하는 subword 3 만 개를 조합해 거의 모든 단어를 표현 .

2

형태론적 일반화

Morphological Generalization

play, playing, played → 같은 어근 "play" 를 공유 . 모델이 어근 의미를 학습 가능 .

3

Vocabulary 크기 제어

Vocabulary Size Control

단어 단위 = 수십만 개로 폭발 . Subword 로 30K 에 묶어 효율적인 임베딩 테이블 유지 .

● 특수 토큰 3 가지 — BERT 의 시그니처

[CLS] · [SEP] · [MASK]

[CLS]

Classification

맨 앞에 무조건 붙는 분류 토큰

- 모든 시퀀스의 첫 번째 토큰 · 최종 hidden vector $C \in \mathbb{R}^H$ · = 문장 전체 요약 벡터 → 분류 태스크

⚠ 주의: NSP 만 학습된 [CLS] 는 fine-tuning 전까지 의미 있는 표현이 아님 (논문 각주 6)

[SEP]

Separator

두 문장을 분리하는 토큰

- 두 sentence 사이 + 각 끝에 위치 · 예 : [CLS] 문장 A [SEP] 문장 B [SEP] · QA, NLI 등 텍스트 쌍 태스크에 필수

⚠ Segment 임베딩과 함께 동작 — 어디까지가 A 이고 B 인지 모델에 알려줌

[MASK]

Mask (사전학습용)

Masked LM 에서 가려진 토큰

- 사전학습 시 토큰의 15% 를 마스킹 · Pre-training ↔ Fine-tuning mismatch · 완화 트릭 : 80% [MASK] / 10% 랜덤 / 10% 원본

⚠ Fine-tuning 시에는 등장하지 않음 — 그래서 위 트릭이 필요함

● 세 임베딩의 합 (Figure 2 핵심)

$$E_{\text{input}} = E_{\text{token}} + E_{\text{segment}} + E_{\text{position}}$$



세 임베딩, 차원 동일

● **Token** (30,000 × 768)

WordPiece vocabulary 룩업

● **Segment** (2 × 768)

A/B 두 벡터만 존재

● **Position** (512 × 768)

학습 가능 — Transformer 와 차이!

Q. 왜 concat 이 아니라 sum?

- Concat = 차원 3 배 → 비효율적
- Sum 도 모델이 직교 부분공간에 분리해 사용

최대 시퀀스 길이 = 512 · Attention 이 $O(L^2)$ 이라 학습 시 90% 는 $L=128$ 로 진행 (부록 A.2)

● 텐서 Shape 의 흐름

Encoder 는 입력 shape 을 보존한다 — 핵심 통찰

Token IDs (입력)	(B, 128)
Token Embedding	(B, 128, 768)
+ Segment Embedding	(B, 128, 768)
+ Position Embedding	(B, 128, 768)
LayerNorm + Dropout	(B, 128, 768)
Encoder Layer × 12	(B, 128, 768)
최종 출력	(B, 128, 768)

핵심 통찰

Transformer Encoder 는 입력 **shape** 을 보존한다

(B, L) → (B, L, H) 한 번 들어 올린 후, 12 층 동안 (B, L, H) 그대로 유지하며 표현만 정교화

최종 출력에서 사용하는 2 가지

C = output[:, 0, :]

[CLS] 위치 (B, 768) · 분류 태스크용

T_i = output[:, i, :]

각 토큰 위치 · NER / QA 등 토큰 레벨

● PyTorch 구현 — 임베딩 레이어

Figure 2 전체를 코드 30 줄로

```
class BERTEmbedding(nn.Module):
    def __init__(self, vocab=30000, H=768,
                  max_len=512, n_seg=2):
        super().__init__()
        # 세 가지 임베딩
        self.token_emb = nn.Embedding(vocab, H)
        self.segment_emb = nn.Embedding(n_seg, H)
        self.position_emb = nn.Embedding(max_len, H)
        self.norm = nn.LayerNorm(H)
        self.dropout = nn.Dropout(0.1)

    def forward(self, input_ids, segment_ids):
        B, L = input_ids.shape
        pos_ids = torch.arange(L).expand(B, L)

        # 세 임베딩의 합 (Figure 2)
        x = (self.token_emb(input_ids)
              + self.segment_emb(segment_ids)
              + self.position_emb(pos_ids))

        x = self.dropout(self.norm(x))
        return x # (B, L, 768)
```

입력 → 출력

input_ids : (B, L)
segment_ids : (B, L)

↓
output : (B, L, 768)

핵심 3 가지

1

nn.Embedding 3 개를
그냥 더한다 (sum)

2

Position ID 는
torch.arange 로 생성

3

LayerNorm → Dropout
순서 (BERT 관례)

● PyTorch 구현 — BERT 전체 골격

임베딩 + Encoder × 12 = 끝

```
class BERT(nn.Module):
    def __init__(self, vocab=30000, H=768,
                  n_layers=12, n_heads=12):
        super().__init__()

        # 1. 임베딩
        self.embedding = BERTEmbedding(vocab, H)

        # 2. Transformer Encoder 스택
        layer = nn.TransformerEncoderLayer(
            d_model=H, nhead=n_heads,
            dim_feedforward=4*H, # = 3072
            activation='gelu', # ⚠ ReLU 아님!
            batch_first=True)
        self.encoder = nn.TransformerEncoder(
            layer, num_layers=n_layers)

    def forward(self, input_ids, segment_ids):
        x = self.embedding(input_ids, segment_ids)
        x = self.encoder(x) # (B, L, 768)
        return x, x[:, 0] # T_i, C
```

⚠ 활성화 함수

Transformer = ReLU

BERT = GELU

Gaussian Error Linear Unit
(Hendrycks & Gimpel, 2016)

결국 이게 다

100 줄

정도면 골격 완성

새 연산 없음.
이미 있는 부품의
영리한 조립.

● Fine-tuning 은 단순하다

사전학습한 BERT 위에 Linear 1 개만 얹으면 끝

분류 태스크 (예 : SST-2)

```
class BERTForClassification(
    nn.Module):
    def __init__(self, bert, K):
        super().__init__()
        self.bert = bert
        self.cls = nn.Linear(768, K)

    def forward(self, ids, seg):
        _, C = self.bert(ids, seg)
        # C: (B, 768)
        return self.cls(C) # (B, K)
```

토큰 레벨 태스크 (예 : NER)

```
class BERTForTokenCls(
    nn.Module):
    def __init__(self, bert, n_tags):
        super().__init__()
        self.bert = bert
        self.cls = nn.Linear(768, n_tags)

    def forward(self, ids, seg):
        T, _ = self.bert(ids, seg)
        # T: (B, L, 768)
        return self.cls(T) # (B, L, n_tags)
```

BERT의 진짜 강점 : 거대 사전학습 모델 위에 **Linear 1 개**만 얹어서 **11 개 NLP 태스크 최고 성능** . Cloud TPU 한 대 · 1 시간 안에 재현 가능 .

정리

오늘 본 4 가지

1

구조

BERT = Transformer Encoder $\times$ 12 (Base)
또는 24 (Large). 1.1 억 / 3.4 억 파라미터.

2

입력 표현

Token + Segment + Position 세 임베딩의 합.
WordPiece 토큰화, [CLS]/[SEP] 로 구조화.

3

Shape 흐름

$(B, L) \rightarrow (B, L, H) \rightarrow$ 모든 레이어 통과하며 (B, L, H) 유지. C 와 T_i 를 활용.

4

구현

PyTorch 100 줄로 골격 완성. 새 연산이 아니라
검증된 부품의 영리한 조립.

그러면 이런 모델을 어떻게 학습시킬까? → *Masked LM · Next Sentence Prediction* 은 다음 발표자에게서